

SYSTEMS AND DATABASE ADMINISTRATION

C21403052



TU Dublin
TU-857/4

Scenario: Stockbroker

1. Security

1.1 Discussion

1.1.1 Major Risks and Challenges

With regards to the stockbroker, the database must manage sensitive financial transactions, historical price data, and portfolio holdings for individuals. This environment poses multiple challenges regarding security and these risks include:

1. Unauthorized Access to trades and funds
2. Remote Access risks for customers
3. Data Theft and manipulation

1.1.2 Measures taken to combat risks and challenges

Ways to mitigate these major risks to the database are:

1. **Role Based Access Control & Principle of Least Privilege** where the two primary roles trader and customer have privileges according to their uses. E.g. trader can insert and update records in the Positions table but only from the office network and the customer can only read their portfolio and price data relevant to their current investments. Separating the logins for each user, ensuring privileges are specified and only granted when needed mitigates the unauthorized access to trades and funds risk.
2. **Network Security and Secure Connections:** Ensuring the system is internal-only for traders, meaning that the PostgreSQL connections are restricted to internal IP range, blocking remote logins. Also, another way to mitigate remote access risks for customers is to enforce secure TLS channels for all external connections from customers, which prevents eavesdropping or man in the middle attacks.
3. **Monitoring and logging** the activity logs and auditing the database to detect queries or modifications.

1.1.3 Recommended post-deployment policies

1.1.4 Measures taken to combat risks and challenges

Ways to mitigate these major risks to the database are:

1. **Periodic Access Review:** The company for the stockbrokers will need to conduct quarterly or biquarterly reviews to ensure no outdated or unnecessary privileges exist that can be exploited for traders or customers.
2. **Patching & Updates:** The company will also need to keep PostgreSQL, and the OS fully patched to protect against all known vulnerabilities to the network.
3. **Encryption:** The use of encryption techniques where feasible for sensitive data such as the Positions data that reflects holdings and the trader's data storing authentication details will also be necessary.

4. **Security Awareness Training:** This will be necessary for the traders and staff to understand how they could be manipulated or used to breach the database and provide access to sensitive information.

1.2 Step-by-step

1.2.1 Creating Roles and setting up Privileges

1. First create roles for the traders and customers.
2. Then Create user accounts for the traders and customers.
3. Grant Privileges to insert and update on Positions to traders
4. Grant read only access to customers on positions, portfolios and prices
5. Grant limit trader connections to an office network e.g. 192.168.56.0/24

```
# TYPE DATABASE USER ADDRESS METHOD
# Traders - only from office LAN (192.168.56.0/24)
host all trader_role 192.168.56.0/24 scram-sha-256
# Customers -anywhere, but have a SSL connection
hostssl all customer_role 0.0.0.0/0 scram-sha-256
```

6. Restart the Postgres to apply the changes

1.2.2 Enabling SSL

1. Go to PostgreSQL.conf file
2. Change ssl = off -> ssl = on
3. And set up the ssl cert file and key file

```
# - SSL -

ssl = on
#ssl_ca_file = ''
ssl_cert_file = 'server.crt'
#ssl_crl_file = ''
#ssl_crl_dir = ''
ssl_key_file = 'server.key'
```

4. Then exit and save out of the postgresql.conf file and create a server cert and key using the following command:
openssl req -nodes -new -x509 -keyout server.key -out server.crt -subj '/C=IE/L=Dublin/O=Stockbrokers/CN=postgres'
5. Then restart the server using: pg_ctl -D /usr/local/pgsql/data restart

1.2.3 Implementing Row Level Security and Password Policies

1. Enable RLS on positions and portfolios
2. Create policy for customers so that they can only see rows that belong to their portfolio on the positions and portfolios tables
3. Set password_encryption to scram-sha-256 in postgresql.conf

2. Auditing

2.1 Discussion

Auditing for the stockbroker database is a critical part for ensuring that it meets the financial regulations set out, for example fraud detection and operational transparency.

Each trade involves potential large investments and sales; therefore, a robust audit trail is needed to:

1. **Regulate Compliance:** Basically, demonstrating that all trades, positions and price updates are monitored to ensure no insider trading or fraud occurs to government bodies.
2. **Security Monitoring:** To detect any unauthorized access or queries that might arise outside of the trade hours
3. **Forensic Analysis:** In the event that a breach or inside attack does occur, the auditing will provide the timeline of any access and changes that could've been executed.

The **objective** of conducting audits is to track any changes to the tables, monitor the access of sensitive information and to record authentication events to view when a failed login attempts occur to try obtaining access.

Configuration to support the auditing process are, pgaudit in PostgreSQL and Native Logging.

The chosen option for auditing is using the PostgreSQL logs for the stockbroker's database as they will need it for transparency and comprehensiveness without implementing excessive overhead. Triggers is another option that could be used for the auditing process, but to me is unnecessary unless extremely fine detailed before and after data changes are required, therefore configuring the logging to a directory in the postgresql.conf file would suffice for the compliance and update detection.

2.2 Step-by-step

1. In /usr/local/pgsql/data create a directory called pg_log
2. Open the postgresql.conf file and edit the:
 - a. Log statement to all
 - b. Log directory to pg_log
 - c. Log filename to postgresql-%Y-%m-%d_%H%M%S.log
 - d. Logging collector to on
 - e. Log min error statement to error
 - f. Log destination to csvlog
3. Save and exit the config file
4. Restart the server
5. Inserted into the tables and viewed logs being generated in the directory

```
postgres@stockbroker:/usr/local/pgsql/data/pg_log$ ls -l
total 68
-rw----- 1 postgres postgres 18806 Apr 23 18:36 postgresql-2025-04-23_181214.csv
-rw----- 1 postgres postgres  166 Apr 23 18:12 postgresql-2025-04-23_181214.log
-rw----- 1 postgres postgres  3437 Apr 23 18:37 postgresql-2025-04-23_183641.csv
-rw----- 1 postgres postgres  166 Apr 23 18:36 postgresql-2025-04-23_183641.log
-rw----- 1 postgres postgres  7851 Apr 23 19:53 postgresql-2025-04-23_183728.csv
-rw----- 1 postgres postgres  166 Apr 23 18:37 postgresql-2025-04-23_183728.log
-rw----- 1 postgres postgres 15922 Apr 23 20:11 postgresql-2025-04-23_195311.csv
-rw----- 1 postgres postgres  3988 Apr 23 20:11 postgresql-2025-04-23_195311.log
postgres@stockbroker:/usr/local/pgsql/data/pg_log$
```

3. Performance Optimisation

Reminder: Questions to answer in this section

What are the most likely performance bottlenecks and potential performance issues in your database?

What options are available to optimise performance?

What have you chosen to implement and why?

What are the benefits and drawbacks of your chosen methods?

3.1 Discussion

Performance optimisation is required to prevent any possible performance bottlenecks and issues from occurring and stalling or slowing down the interactions within the database. The most likely performance bottlenecks for this database are:

1. Frequent Position table being updated and causing a high frequency of updates to be maintained and updated across to all traders
2. Large Historical Price table that grows daily, storing the closing prices of potential thousands of companies over time.
3. Frequent Reads by customers pulling the same data from positions or prices concurrently can lead to potential I/O saturation.

There are a few potential options for optimization:

1. Indexing: B-tree indexing on frequently filtered columns to speed up queries and updates.
2. Partitioning: Partitioning the price table by date to improve performance for historical queries and speed up the archiving of old data
3. Connection Pooling: pgBouncer can handle the bursts of concurrent connections without overloading the server

For this database the method chosen for performance optimization are partitioning the price table to give exponential growth, and to partition by quarter to prevent large single-table scans. The other method chosen is the B-tree indexing of positions (`portfolio_id`), prices (`company_id`, `date`), and companies (`internal_id`) for typical queries.

The benefits of these chosen methods are that partitioning provides easier maintenance, and faster queries targeting for specific date ranges, and indexing will provide significant query performance improvements.

The potential drawbacks of implementing these methods are that partitioning can require more admin overhead for complex schemas and indexing can produce slower write to the database tables due to index maintenance and increased storage.

3.2 Step-by-step

1. First add the indexes for the prices, positions and companies

2. Then partition the prices table by date

```
partition_table_name
-----
prices
(1 row)
```

3. Create quarterly partitions for the year 2025

Schema	Name	Type	Owner	Table
public	companies_companyname_key	index	postgres	companies
public	companies_pkey	index	postgres	companies
public	idx_positions_portfolio	index	postgres	positions
public	idx_prices_date	partitioned index	postgres	prices
public	portfolios_pkey	index	postgres	portfolios
public	portfolios_portfolioname_key	index	postgres	portfolios
public	positions_pkey	index	postgres	positions
public	prices_2025_q1_companyid_date_idx	index	postgres	prices_2025_q1
public	prices_2025_q1_pkey	index	postgres	prices_2025_q1
public	prices_2025_q2_companyid_date_idx	index	postgres	prices_2025_q2
public	prices_2025_q2_pkey	index	postgres	prices_2025_q2
public	prices_2025_q3_companyid_date_idx	index	postgres	prices_2025_q3
public	prices_2025_q3_pkey	index	postgres	prices_2025_q3
public	prices_2025_q4_companyid_date_idx	index	postgres	prices_2025_q4
public	prices_2025_q4_pkey	index	postgres	prices_2025_q4
public	prices_pkey	partitioned index	postgres	prices
public	traders_pkey	index	postgres	traders

4. Backup/Recovery/Availability

4.1 Discussion

The objective of implementing backups/recovery policies is that it:

1. Will safeguard the Financial data of the stockbrokers
2. Regulatory Complies with the maintaining of transaction logs for auditing by financial authorities
3. Can minimize the downtime if the database ever went down by quickly restoring the data during active trading hours

There are several options available for implementing backup/recovery policies:

1. Pg_dump for simplified table-level recovery but has the disadvantage of being potentially slow for the large volumes of historical price data.
2. Physical Backups using pg_basebackup and a WAL archiving for faster restoration of the database and enabling point in time recovery.
3. Snapshot-Based Backup using filesystem level backup which is quick but must ensure PostgreSQL consistency.

For this database the chosen approach for backup/recovery is:

1. Physical Backups using the pg_basebackup which is ideal for the large size of Prices table and the need for point-in-time recovery for the stockbrokers. This will ensure minimal data loss in disaster scenarios.

The benefits of using this method of backup and recovery are that the physical backups are faster to restore and provide the point in time recovery which is required for the financial authorities.

The drawbacks however of using these methods are that the physical backups require additional storage for the WAL archives and is more complex to maintain.

4.2 Step-by-step

1. First made a directory called wal_archive in the /usr/local/pgsql/data
2. Gave the permissions of the directory to the postgres user
3. Open the postgresql.conf file and edit the following lines:
 - a. Archive mode to on
 - b. Archive command to 'cp %p /wal_archive/%f'
 - c. Archive timeout to 60
4. Then restart the database and test it by performing a backup and a PITR